

1. Objetivo

O objetivo do projeto foi fazer o cálculo do Conjunto de Mandelbrot na linguagem C, fazendo quatro versões diferentes do programa: uma serial, uma com OpenMP e duas utilizando Pthreads com estratégias diferentes da divisão do trabalho. Também foi feito o registro do tempo de execução de cada versão e a comparação dos resultados para verificar se todas geravam a mesma imagem.

2. Checklist de Requisitos

Um requisito por linha. Sem parágrafo, se precisar explicar algo, use a coluna "como testar".

Implementação serial	OK	./mandelbrot 800 600 1000 8
Implementação com OpenMP	OK	./openMP 800 600 1000 8
Pthreads1	OK	./pthreads1 800 600 1000 8
Pthreads2	OK	./pthreads2 800 600 1000 8
Geração das imagens .pgm	OK	Verificar os arquivos gerados
imagens iguais nas quatro versões	OK	Cmp entre os arquivos .pgm
Medição do tempo de execução	OK	Ver o times.txt
Tratamento de argumentos inválidos	OK	Testes com argumentos inválidos e registro das mensagens em erros.txt
Compilação pelo Makefile	OK	make
Readme	OK	Arquivo readme.md

3. Como Reproduzir

Compilar:

```
make clean
make
```

Executar a versão serial:

```
./mandelbrot 800 600 1000 8
```

Executar a versão com OpenMP:

```
./openMP 800 600 1000 8
```

Executar a versão Pthreads 1:

```
./pthreads1 800 600 1000 8
```

Executar a versão Pthreads 2:

```
./pthreads2 800 600 1000 8
```

4. Arquitetura

Arquivos e responsabilidades:

1. src/mandelbrot.c - responsável pela implementação serial do cálculo do Conjunto de Mandelbrot.
2. src/openMP.c - responsável pela implementação utilizando OpenMP para dividir o cálculo dos pixels entre as threads.
3. src/pthreads1.c - responsável pela implementação com Pthreads utilizando a primeira estratégia de divisão do trabalho.
4. src/pthreads2.c - implementação em que o cálculo do Mandelbrot é feito de forma serial e as threads são utilizadas posteriormente para processar a imagem gerada, trabalhando com diferentes faixas de intensidade dos pixels.

Decisões técnicas:

1. Usei malloc para criar o espaço necessário para armazenar os pixels da imagem.
2. Usei clock_gettime para medir o tempo de execução de cada implementação e salvar os resultados no times.txt.
3. Na versão OpenMP, usei #pragma omp parallel for para paralelizar o cálculo dos pixels.
4. Nas versões com Pthreads, usei pthread_create e pthread_join para criar as threads e esperar que elas terminassem suas tarefas.
5. As quatro versões salvam os pixels no formato .pgm, permitindo comparar os arquivos e verificar se todas as implementações geraram o mesmo resultado.

5. Estratégias e Diário de Desenvolvimento

O desenvolvimento do projeto foi feito por partes, primeiro fazendo a versão serial do Conjunto de Mandelbrot e depois passando para as versões paralelas. Eu preferi fazer uma implementação funcionar e testar antes de começar a próxima.

Primeiro implementei a versão serial, fazendo o cálculo dos pixels e salvando o resultado em um arquivo. Também fiz alguns testes com imagens menores para verificar se o arquivo estava sendo gerado corretamente. Depois testei uma imagem maior, verificando a quantidade de linhas e de pixels por linha.

Depois comecei a implementação com OpenMP. Nessa parte tive alguns problemas no Makefile e também um erro na posição do clock_gettime em relação ao #pragma omp parallel for. O erro apareceu na compilação e precisei ajustar a organização do código para conseguir compilar.

Depois disso comecei as versões com Pthreads. No começo o makefile ainda não estava configurado para gerar todos os executáveis, então algumas tentativas de executar pthreads1 falharam porque o executável ainda não existia. Ajustei o Makefile e depois consegui compilar a versão normalmente.

Depois que as quatro versões estavam funcionando, comecei a fazer os testes de comparação entre as imagens e os testes de desempenho. Também foram feitos testes com diferentes quantidades de threads para observar como o tempo mudava conforme aumentava o número de threads. Fiz os testes de tratamento de erros, verificando argumentos faltando, argumentos a mais, valores negativos, zero, textos no lugar de números e números muito grandes. A ideia foi verificar não só se o programa funcionava no caso normal, mas também se apresentava erro sem quebrar a execução.

Depois que as quatro versões estavam funcionando, alterei o tratamento de erros para que as mensagens fossem registradas no arquivo erros.txt. A mesma função foi adicionada às quatro implementações, mantendo o comportamento de validação dos argumentos.

Também foi ajustada a estratégia do Pthreads2. Em vez de utilizar as threads para realizar diretamente o cálculo do Mandelbrot, a versão passou a calcular a imagem de forma serial e utilizar as threads em uma etapa

posterior para processar os pixels de acordo com suas faixas de intensidade.

5.1 Estratégias da semana

S1	Fazer uma versão por vez	Comecei pela versão serial e só depois fui para OpenMP e Pthreads	Nao teve
S2	Testar antes de continuar	Depois de implementar cada versão eu executava testes para ver se estava funcionando antes de começar a próxima	Nao teve
S3	Comparar as imagens	Usei cmp para comparar os arquivos gerados pelas versões paralelas com a versão serial	Nao teve
S4	Testar diferentes threads	Executei as versões paralelas com 1, 2, 4 e 8 threads para comparar os tempos	Nao teve
S5	Tratar os erros no final	Primeiro foquei em fazer as implementações funcionarem e depois fiz os testes de argumentos inválidos e tratei os erros	Nao teve

5.2 Diário de Tentativas

Uma linha por tentativa relevante, marcando a qual estratégia (S1/S2/S3) ela pertence, obviamente precisa ser em ordem cronológica. A coluna "Quando" precisa bater com a ordem cronológica do evidencias.log, é isso que comprova que o fluxo é real. "Hipótese/Causa" só é preenchida quando o resultado foi falha ou parcial.

1	S1	Tentei utilizar o make no começo do projeto	falhou	O Makefile ainda não estava configurado corretamente	26/08	No targets
2	S1	Compilei e executei a versão serial	ok		26/08	

						gcc e execução do mandelbro t
3	S2	Testei a geração da imagem serial com 10x10	ok		26/08	arquivo pgm gerado
4	S1	Comecei a implementação com OpenMP	FALHOU	O clock_gettime estava em uma posição errada em relação ao pragma	31/08	erro na compilação
5	S1	Corrigi o OpenMP e compilei novamente	OK		31/08	make compilou mandelbro t e openMP
6	S1	Tentei adicionar Pthreads1 ao Makefile	FALHOU	O target do pthreads1 estava configurado errado	31/08	erro do make
7	S3	Comparei as imagens das quatro implementações	OK		01/09	Cmp sem apresentar diferenças
8	S4	Testei as versões com diferentes quantidades de threads	OK		01/09	testes com 1, 2, 4 e 8 threads
9	S5	Testei argumentos inválidos, zero, negativos e números muito	OK		01/09	mensagens de erro apresentada corretamente

		grandes				
10	S1	Altere a estratégia do Pthreads2	ok	O cálculo passou a ser feito de forma serial e as threads atuam posteriormente	02/09	Execução e comparação das imagens
11	S5	Altere o tratamento de erros para registrar as mensagens em erros.txt	OK	Foi criada uma função para registrar os erros	02/09	Testes com argumentos inválidos

6. Evidências

6.1 Log automático (faz o trabalho pesado por você)

No início da sessão de trabalho, rode:

```
script -a evidencias.log
```

Isso grava automaticamente todos os comandos e saídas do terminal num arquivo, sem você precisar tirar print de cada linha. Comece a sessão com date, whoami e pwd, assim o log já tem timestamp, usuário e diretório sem esforço extra. **O evidencias.log deve ser enviado dentro do .tar**

6.2 Prints

- Erro principal: erro de compilação na implementação do OpenMP causado pela posição do clock_gettime em relação ao #pragma omp parallel for.

Obs: eu nao tenho prints, mas a parte do evidencias.log que noticia isso é entre a linha 255 e 275.

```
[?2004h]0;luisf@aluisio:
/mnt/c/Users/luisf/FACULDADE/lfass-2/Conjunto-de-Mandelbrot[01;32mluisf@aluis
io[00m:[01;34m/mnt/c/Users/luisf/FACULDADE/lfass-2/Conjunto-de-Mandelbrot[00m
$ [7mcmp mandelbrot_lfass_serial.pgm mandelbrot_lfass_openMP.pgm[27mcmp
mandelbrot_lfass_serial.pgm mandelbrot_lfass_openMP.pgm
[?2004l
[?2004h]0;luisf@aluisio:
/mnt/c/Users/luisf/FACULDADE/lfass-2/Conjunto-de-Mandelbrot[01;32mluisf@aluis
io[00m:[01;34m/mnt/c/Users/luisf/FACULDADE/lfass-2/Conjunto-de-Mandelbrot[00m
$ [7mcmp mandelbrot_lfass_serial.pgm mandelbrot_lfass_pthreads1.pgm[27mcmp
mandelbrot_lfass_serial.pgm mandelbrot_lfass_pthreads1.pgm
[?2004l
```

```
[?2004h]0;luisf@aluisio:
/mnt/c/Users/luisf/FACULDADE/lfass-2/Conjunto-de-Mandelbrot[01;32mluisf@aluis
io[00m:[01;34m/mnt/c/Users/luisf/FACULDADE/lfass-2/Conjunto-de-Mandelbrot[00m
$ make clean

[?2004l

rm -f mandelbrot openMP pthreads1

rm -f *.pgm times.txt

[?2004h]0;luisf@aluisio:
/mnt/c/Users/luisf/FACULDADE/lfass-2/Conjunto-de-Mandelbrot[01;32mluisf@aluis
io[00m:[01;34m/mnt/c/Users/luisf/FACULDADE/lfass-2/Conjunto-de-Mandelbrot[00m
$ [?2004l

exit

Script started on 2026-09-01 09:23:33-03:00 [TERM="xterm-256color"
TTY="/dev/pts/2" COLUMNS="141" LINES="9"]

[?2004h]0;luisf@aluisio:
/mnt/c/Users/luisf/FACULDADE/lfass-2/Conjunto-de-Mandelbrot[01;32mluisf@aluis
io[00m:[01;34m/mnt/c/Users/luisf/FACULDADE/lfass-2/Conjunto-de-Mandelbrot[00m
$ [7m./mandelbrot 800 600 1000 4[27m./mandelbrot 800 600 1000 4
```

7. Uso de IA

Usei IA para tirar dúvidas durante a implementação e entender erros de compilação e execução. Também usei para ajudar a definir testes para verificar as quatro versões do programa e comparar os resultados.

Prompts principais:

- A compilação do OpenMP esta dando errado, esta aparecendo isso no terminal, como faço paara resolver?
- O que é pthread?
- Comando para comparar as imagens agora
- Me mande testes varios testes para os 4 calculos para eu ver se meus tratamentos estao certos
- Pra que usar flag no makefile?

O que validei manualmente e como:

Validei manualmente compilando o projeto com make e executando as quatro versões com diferentes parâmetros. Comparei os arquivos .pgm usando cmp para verificar se as imagens eram iguais e conferi os tempos registrados no times.txt. Também testei argumentos inválidos, como texto, zero, números negativos, números muito grandes e quantidade errada de argumentos, verificando se o programa apresentava os erros esperados.

8. Reflexão Final

Com mais tempo, eu melhoraria principalmente a implementação do pthreads2, buscando uma estratégia que

aproveitasse melhor o paralelismo. Uma das principais decisões técnicas foi dividir o cálculo entre as threads no OpenMP e no Pthreads1. Também foi importante utilizar o `clock_gettime` para medir os tempos e o `cmp` para garantir que as imagens fossem iguais. Durante o desenvolvimento, tive alguns erros de compilação relacionados ao Makefile e à organização do código OpenMP. Na próxima vez, eu configuraria o Makefile e a estrutura dos arquivos antes de começar os testes. Também faria testes menores desde o início para encontrar os erros mais rapidamente. Por fim, faria mais execuções ainda para obter médias dos tempos e uma comparação mais confiável. Os testes com diferentes quantidades de threads mostraram que o tempo tende a diminuir no OpenMP e no Pthreads1 porque o trabalho é dividido entre as threads, mas o ganho não é proporcional ao número de threads devido aos custos de criação e sincronização. No Pthreads2, percebi que aumentar a quantidade de threads não trouxe o mesmo ganho, pois o cálculo principal continua sendo realizado de forma serial.

9. Checklist Final de Entrega

Compilei do zero seguindo só o que está no relatório	sim
Rodei os testes e evidencias.log foi gerado	sim
Tenho prints obrigatórios	So tenho o log, esqueci dos prints do terminal.
Testei pelo menos um caso limite e um caso inválido	sim
Preenchi a seção de uso de IA	sim
Revisei o relatório e removi frases genéricas / vazias	sim

10. Se eu tivesse mais 2 horas

Se eu tivesse mais 2 horas, faria mais testes de desempenho, repetindo cada execução para calcular uma média dos tempos.

Também corrigiria qualquer warning restante na compilação para deixar o código mais limpo.

Testaria casos limites com valores maiores de largura, altura e número de iterações.

Melhoraria a robustez do tratamento de erros, principalmente em situações de falha na criação das threads. Tentaria melhorar a estratégia do Pthreads2 para aproveitar melhor o paralelismo. Também revisaria o código dos quatro programas para diminuir repetições e melhorar a organização. Por fim, faria uma última compilação do zero e repetiria os testes principais para garantir que tudo continuasse funcionando.